

Jenkins AWS ECR Setup Guide

This guide explains how to configure Jenkins to push Docker/Podman images to AWS Elastic Container Registry (ECR).

Prerequisites

- Jenkins installed on Windows
 - AWS CLI installed on the Jenkins agent machine
 - Podman installed and configured
 - AWS account with appropriate permissions
-

1. Install AWS CLI on Jenkins Agent

If AWS CLI is not installed, download and install it:

1. Download AWS CLI from: <https://awscli.amazonaws.com/AWSCLIV2.msi>
2. Run the installer
3. Verify installation by opening PowerShell and running:

```
4. aws --version
```

2. Get AWS Access Key ID and Secret Access Key

Option A: Create New IAM User (Recommended)

1. **Log in to AWS Console** at <https://console.aws.amazon.com>
2. Navigate to **IAM** (Identity and Access Management)
3. Click **Users** in the left sidebar
4. Click **Create user**
5. Enter a username (e.g., jenkins-ecr-user)
6. Click **Next**
7. **Attach policies directly:**
 - Search for and select **AmazonEC2ContainerRegistryPowerUser**
 - Or create a custom policy with these permissions:

```
○ {  
○   "Version": "2012-10-17",  
○   "Statement": [  
○     {  
○       "Effect": "Allow",  
○       "Action": "ecr:*",  
○       "Resource": "*"
```

```

    ○      "Effect": "Allow",
    ○      "Action": [
    ○          "ecr:GetAuthorizationToken",
    ○          "ecr:BatchCheckLayerAvailability",
    ○          "ecr:InitiateLayerUpload",
    ○          "ecr:UploadLayerPart",
    ○          "ecr:CompleteLayerUpload",
    ○          "ecr:PutImage"
    ○      ],
    ○      "Resource": "*"
    ○    }
  ○  ]
○  }

```

8. Click **Next**, then **Create user**
9. Click on the created user, then go to **Security credentials** tab
10. Scroll to **Access keys** section
11. Click **Create access key**
12. Select **Application running outside AWS**
13. Click **Next**, optionally add a description tag
14. Click **Create access key**
15. **IMPORTANT:** Copy both:
 - **Access key ID** (e.g., AKIAIOSFODNN7EXAMPLE)
 - **Secret access key** (e.g., wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY)
16. **Save these securely** - the secret key won't be shown again!

Option B: Use Existing IAM User

1. Go to **IAM** → **Users**
2. Select your existing user
3. Go to **Security credentials** tab
4. Under **Access keys**, click **Create access key**
5. Follow steps 12-16 above

3. Create ECR Repository

Option A: Using AWS Console

1. Log in to **AWS Console**
2. Navigate to **Amazon ECR** (Elastic Container Registry)
3. Click **Get Started** or **Create repository**
4. Select **Private** repository
5. Enter repository name: test-ems-api

6. Leave other settings as default (or configure as needed)
7. Click **Create repository**
8. Note your repository URL: <account-id>.dkr.ecr.<region>.amazonaws.com/test-ems-api

Option B: Using AWS CLI

Open PowerShell and run:

```
# Set your region
$AWS_REGION = "us-east-1"

# Create repository
aws ecr create-repository --repository-name test-ems-api --region $AWS_REGION

# Optional: Add image scanning
aws ecr put-image-scanning-configuration `
  --repository-name test-ems-api `
  --image-scanning-configuration scanOnPush=true `
  --region $AWS_REGION

# Optional: Set lifecycle policy to keep only last 10 images
aws ecr put-lifecycle-policy `
  --repository-name test-ems-api `
  --lifecycle-policy-text '{
    "rules": [{
      "rulePriority": 1,
      "description": "Keep last 10 images",
      "selection": {
        "tagStatus": "any",
        "countType": "imageCountMoreThan",
        "countNumber": 10
      },
      "action": { "type": "expire" }
    }],
  }' `
  --region $AWS_REGION
```

4. Install Jenkins Plugins

1. Go to **Jenkins Dashboard**

2. Click **Manage Jenkins** → **Manage Plugins**
 3. Go to **Available plugins** tab
 4. Search for and install:
 - **Pipeline: AWS Steps** (required for AWS credentials binding)
 - **CloudBees AWS Credentials** (should be installed as dependency)
 5. Click **Install without restart** or **Download now and install after restart**
 6. Restart Jenkins if required
-

5. Configure AWS Credentials in Jenkins

1. Go to **Jenkins Dashboard**
 2. Click **Manage Jenkins** → **Manage Credentials**
 3. Click on **(global)** domain
 4. Click **Add Credentials** in the left sidebar
 5. Fill in the form:
 - **Kind:** Select **AWS Credentials**
 - **Scope:** Global (Jenkins, nodes, items, all child items, etc)
 - **ID:** aws-credentials (must match the credentialsId in Jenkinsfile)
 - **Description:** AWS ECR Credentials (optional)
 - **Access Key ID:** Paste your AWS Access Key ID
 - **Secret Access Key:** Paste your AWS Secret Access Key
 6. Click **Create**
-

6. Update Jenkinsfile Variables

Edit the Jenkinsfile and update these environment variables:

```
environment {  
    AWS_REGION = 'us-east-1'           // Change to your AWS region  
    AWS_ACCOUNT_ID = '123456789012'    // Change to your 12-digit AWS account  
    ID  
    ECR_REPOSITORY = 'test-ems-api'     // Change if you used a different name  
    IMAGE_TAG = "${env.BUILD_NUMBER}"  
    ECR_REGISTRY = "${AWS_ACCOUNT_ID}.dkr.ecr.${AWS_REGION}.amazonaws.com"  
    IMAGE_NAME = "${ECR_REGISTRY}/${ECR_REPOSITORY}"  
}
```

How to Find Your AWS Account ID

Method 1: AWS Console

- Log in to AWS Console
- Click on your username in the top right
- Your 12-digit account ID is displayed

Method 2: AWS CLI

```
aws sts get-caller-identity --query Account --output text
```

7. Test the Pipeline

1. Commit and push the updated Jenkinsfile to your repository
2. Go to your Jenkins job
3. Click **Build Now**
4. Monitor the console output

The pipeline will:

- Check out the code
 - Install dependencies
 - Build the Docker image with Podman
 - Authenticate to AWS ECR
 - Push the image to ECR with two tags: <build-number> and latest
-

8. Verify Image in ECR

Using AWS Console

1. Go to **Amazon ECR** in AWS Console
2. Click on your repository (test-ems-api)
3. You should see your images with tags

Using AWS CLI

```
aws ecr describe-images --repository-name test-ems-api --region us-east-1
```

Troubleshooting

AWS CLI Not Found

- Ensure AWS CLI is installed and added to system PATH
- Restart Jenkins service after installing AWS CLI
- Check with: `aws --version`

Authentication Errors

- Verify AWS credentials are correct in Jenkins
- Check IAM user has ECR permissions
- Ensure credential ID in Jenkinsfile matches: `aws-credentials`

Repository Does Not Exist

- Create the ECR repository first (see Section 3)
- Verify repository name matches `ECR_REPOSITORY` variable

Podman Login Fails

- Check that AWS CLI can authenticate: `aws ecr get-login-password --region us-east-1`
- Verify the ECR registry URL is correct
- Ensure Podman is running and accessible

Permission Denied Errors

Ensure your IAM user/role has these ECR permissions:

- `ecr:GetAuthorizationToken`
- `ecr:BatchCheckLayerAvailability`
- `ecr:InitiateLayerUpload`
- `ecr:UploadLayerPart`
- `ecr:CompleteLayerUpload`
- `ecr:PutImage`

Security Best Practices

1. **Use IAM Roles** (if Jenkins is on EC2): Instead of access keys, attach an IAM role to the EC2 instance
2. **Rotate Access Keys**: Regularly rotate AWS access keys
3. **Least Privilege**: Only grant necessary ECR permissions
4. **Enable Image Scanning**: Use ECR's built-in vulnerability scanning
5. **Use Lifecycle Policies**: Automatically clean up old images to save costs
6. **Enable Audit Logging**: Use AWS CloudTrail to log ECR API calls

Additional Resources

- [AWS ECR Documentation](#)
- [Jenkins AWS Steps Plugin](#)
- [AWS CLI ECR Commands](#)
- [IAM Best Practices](#)

Options to run as container

1) Run using Podman (Windows / Linux)

```
podman run -it --rm \  
  -p 3000:3000 \  
  --name api \  
  <ACCOUNT_ID>.dkr.ecr.<REGION>.amazonaws.com/<REPO>:<TAG>
```

Optional switches:

```
-d                # run detached  
--env KEY=value   # set environment variables  
--env-file env.txt # load env from file  
--volume host:ctr # mount volume  
--restart=always  # restart policy  
--pull=always     # always pull new image
```

2) Run using Docker

```
docker run -it --rm \  
  -p 3000:3000 \  
  --name api \  
  <ACCOUNT_ID>.dkr.ecr.<REGION>.amazonaws.com/<REPO>:<TAG>
```

Optional:

```
-d  
--env KEY=value  
--env-file env.txt  
--volume host:ctr  
--restart always  
--pull always
```

3) Run with Podman (systemd-ready)

```
podman generate systemd --name api --files
```

4) Run using AWS ECS (EC2 or Fargate)

Task Definition → Container Image: <ECR_URI>

5) Run using AWS Lambda (container image)

Lambda → Create Function → Container Image → Select from ECR

6) Run using AWS App Runner

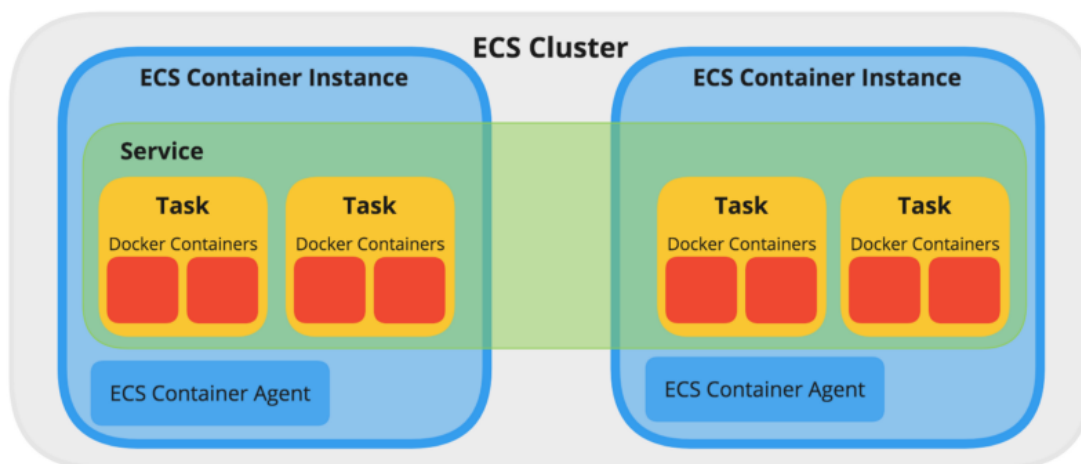
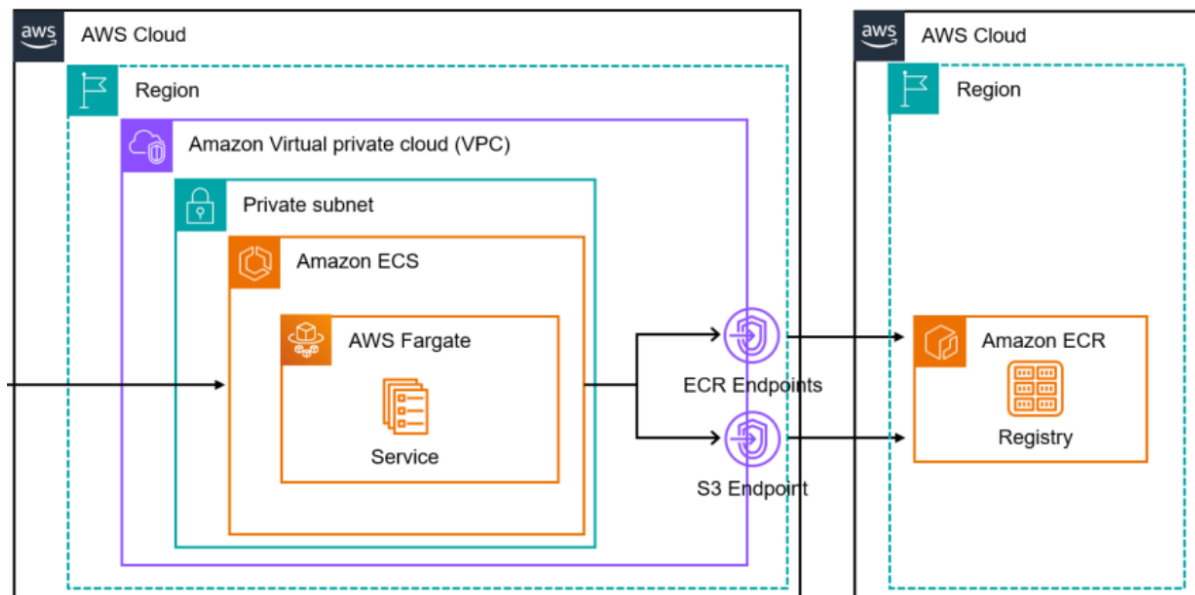
App Runner → Create Service → Source: Container Registry → ECR

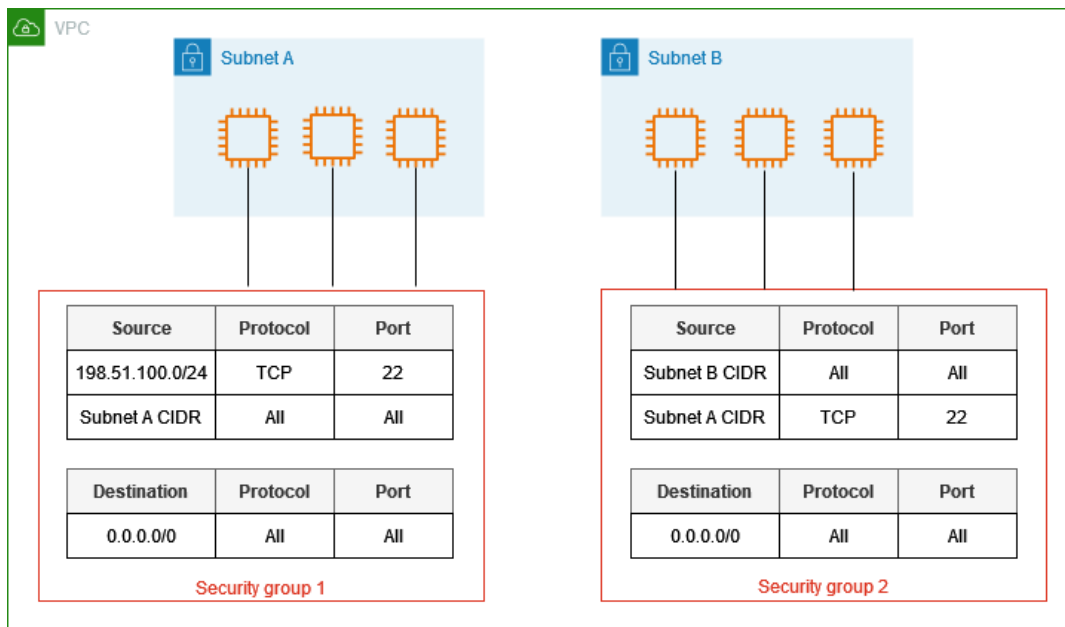
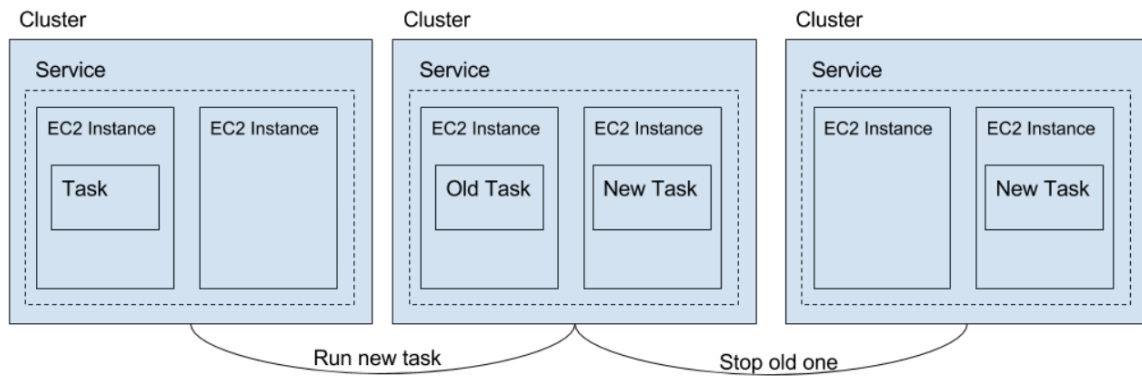
7) Run using AWS EKS / Kubernetes

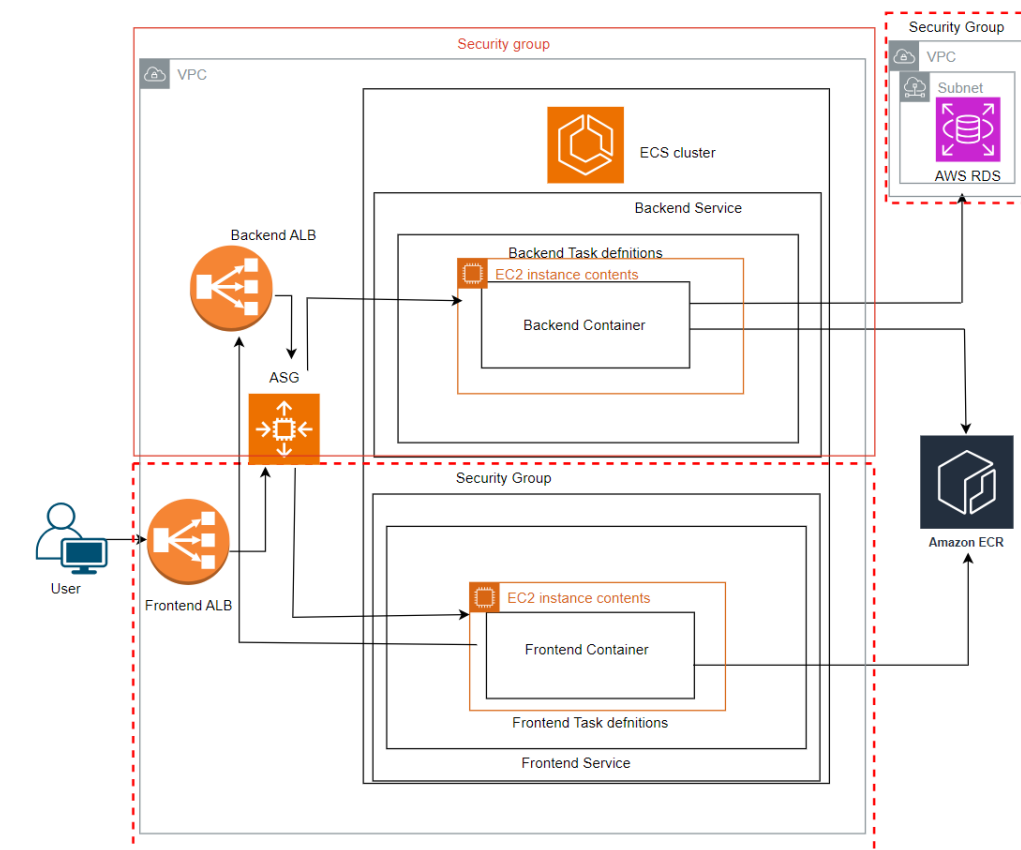
image: <ACCOUNT_ID>.dkr.ecr.<REGION>.amazonaws.com/<REPO>:<TAG>

Run an ECR Image on ECS Fargate

AWS ECS (Elastic Container Service) is a fully managed **container orchestration service** that simplifies the deployment, management, and scaling of containerized applications. It allows you to run applications using containers like Docker without the complexity of managing the underlying infrastructure, and it offers both serverless (AWS Fargate) and instance-based (EC2 instances) compute options







STEP 1 — Go to ECS

1. Open **AWS Console**
2. **IAM** user must have role: **CreateServiceLinkedRole**
3. Search for **ECS (Elastic Container Service)**
4. Click **Create Cluster**

STEP 2 — Create a Cluster

Choose:

- **Networking only (Fargate)** → Continue
- Name: `ems-fargate-cluster`
- Leave defaults
- Click **Create**

STEP 3 — Create a Task Definition (Fargate)

1. Go to **Task Definitions** → **Create new Task Definition**
 2. Choose:
 - **Launch type:** Fargate
 3. **Task Definition Settings**
 - **Name:** `my-api-task`
 - **Task Role:** None (unless your app needs AWS access)
 - **Execution Role:** `ecsTaskExecutionRole`
(create automatically if not present)
 4. **Task Size**
 - **CPU:** `0.25` vCPU
 - **Memory:** `0.5` GB (or as required)
-

STEP 4 — Add Container to Task Def

Click **Add container**:

Container settings:

- **Container name:** `api`
- **Image URI:**
`797721580785.dkr.ecr.us-east-1.amazonaws.com/test-ems-api:latest`
- **Port mappings:**
 - **Container port:** `3000` (or your app port)
 - **Protocol:** TCP

(Your app MUST listen on that port inside container.)

Optional:

- **Environment variables**
- **Log configuration:** `awslogs` (default is fine)

Click **Add** → **Create**

STEP 5 — Run the Task as a Service

1. Go to **Clusters**
2. Click your cluster: `my-fargate-cluster`
3. Go to **Services** → **Create**

Service settings:

- ✓ **Launch type:** Fargate
- ✓ **Task definition:** `my-api-task` (latest revision)
- ✓ **Service name:** `my-api-service`
- ✓ **Service type:** Replica
- ✓ **Number of tasks:** 1

Click Next

STEP 6 — Configure Networking

Select:

- **VPC:** your default VPC
- **Subnets:** select 2 public subnets
- **Security group:**
 - Create new or use existing
 - **Allow inbound port 3000** (or your app port)
- **Assign public IP:** ENABLED
(required unless you use a Load Balancer)

Click Next → Next → Create Service

STEP 7 — Confirm Service is Running

Go to:

- Cluster → **Tasks**

You should see:

- **RUNNING** status
- A public ENI attached (if public IP enabled)

Click the task → find the **Public IP**.

STEP 8 — Test Your API

Open:

`http://<public-ip>:3000`

Your container from ECR should now be running on ECS Fargate.

Database in production

- **Amazon RDS for Oracle (recommended for managed DB)**

Managed backups, Multi-AZ, automated maintenance. Consider licensing (License Included vs Bring Your Own License).

- **Self-managed Oracle on EC2**

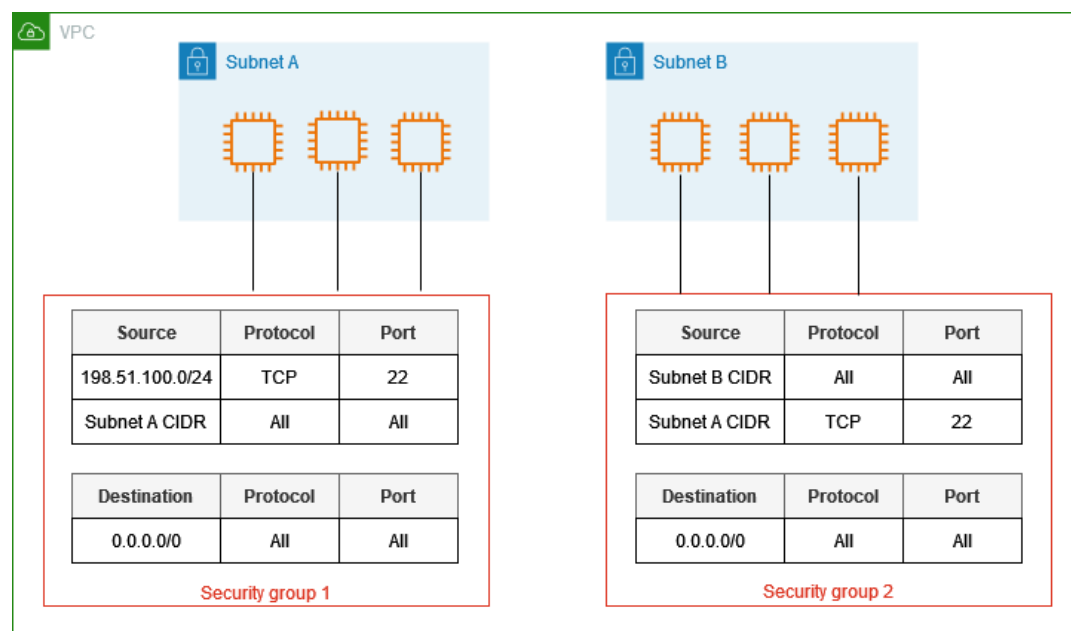
Full control (install Oracle, attach EBS for persistence). You must manage backups, HA, patching.

- **Oracle DB outside AWS (Oracle Cloud or other)**

Works if latency and network/VPN are acceptable.

- **Oracle in a container (NOT recommended for production)**

Oracle provides images (XE) but containers are ephemeral — you still need persistent storage and it's unsupported for many production scenarios. Avoid on Fargate.



Connect API (from ECS fargate) to RDS (oracle db)

STEP 1 — Find your Oracle RDS endpoint

1. Open **AWS Console** → **RDS**
2. Click your Oracle DB instance
3. Under **Connectivity & security**
 - Copy **Endpoint** (looks like: yourdb.xxxxxxx.us-east-1.rds.amazonaws.com)
 - Note the **Port** (default: 1521)
 - Note Service name / SID:
 - For RDS Oracle, **service name = ORCL** by default (unless you created a custom PDB)

Replace your local values:

Local	AWS Value
DB_HOST=oracledb	DB_HOST=RDS endpoint
DB_SERVICE_NAME=XEPDB1	Usually ORCL (unless configured otherwise)

STEP 2 — Update RDS Security Group to allow ECS tasks (important!)

Your Fargate task must be allowed to reach Oracle DB port **1521**.

1. Go to EC2 → Security Groups

2. Find the SG attached to your RDS instance

(Under RDS → DB name → Connectivity → VPC Security groups)

3. Edit Inbound rules

Add:

Type	Port	Source
Custom TCP	1521	Security group of ECS tasks

This is the MOST important step.

How to find your ECS Task Security Group

1. Go to **ECS** → **Clusters** → **default** → **Services** → (your service)
2. Open the service → **Network configuration**
3. Note the **Security Group** (e.g. sg-0abcd1234)

Use this SG as the **source** for the RDS inbound rule.

NOT 0.0.0.0/0, not your IP.

ECS tasks connect using ENI inside the VPC, via this SG.

STEP 3 — Store DB credentials in AWS Secrets Manager

(Recommended and secure. Do NOT store in plain-text env vars.)

1. Go to **AWS Console** → **Secrets Manager**
2. Click **Store a new secret**
3. Secret type: **Other type of secret**
4. Add key/value pairs:

```
DB_HOST=your-rds-endpoint
DB_PORT=1521
DB_USERNAME=training
DB_PASSWORD=training123
DB_SERVICE_NAME=ORCL
DB_TYPE=oracle
```

5. Name the secret:

ems-api/db-creds

6. Click **Next** → **Next** → **Store**

Copy the **Secret ARN**.

STEP 4 — Grant ECS Task Role permission to read the secret

1. Go to **IAM** → **Roles**
2. Open your **task role** (not execution role):
 - Usually named something like: `ecsTaskRole` or created by you

3. Click **Add permission** → **Attach policies**
4. Search: **SecretsManagerReadWrite** (or create least privilege later)
5. Attach it

Later you can restrict to only that specific secret.

STEP 5 — Inject DB secrets into your ECS Task Definition

1. Go to **ECS** → **Task Definitions**
2. Open your Task Definition → Create new revision
3. Under **Container definition** → Environment variables
4. Scroll to **Secrets**
5. Click **Add secret**

Add:

ECS Environment Var	Value From
DB_HOST	select secret key DB_HOST
DB_PORT	DB_PORT
DB_USERNAME	DB_USERNAME
DB_PASSWORD	DB_PASSWORD
DB_SERVICE_NAME	DB_SERVICE_NAME
DB_TYPE	DB_TYPE

Pick each value from the secret you created.

6. Save → Create new revision
7. Update your ECS Service to use this revision
 - ECS → Cluster → Service → Update → Select new revision

STEP 6 — Redeploy ECS Service

1. Go to **ECS** → **Cluster** → **your service**
2. Click **Update**
3. Choose your **new Task Definition revision**
4. Save & deploy

Wait for new task.

STEP 7 — Verify connectivity

Check CloudWatch logs:

1. ECS → your service → your task
2. Click **Logs**
3. Look for:
 - “Connected to Oracle”
 - Or errors like “ORA-xxxx”, “getaddrinfo ENOTFOUND”, “ECONNREFUSED”

If you still get connection errors → 99% chance it's **security group** rules.

Your final expected environment inside ECS

```
DB_TYPE=oracle
DB_HOST=<your-rds-endpoint>
DB_PORT=1521
DB_USERNAME=training
DB_PASSWORD=training123
DB_SERVICE_NAME=ORCL
DB_SYNCHRONIZE=true
```

(If you manually changed the Oracle PDB to XEPDB1, then use that instead.)